

Atividade 03 — Protótipo individual com ESP32 e Wokwi

Aluno: João Pedro de Brito Tomé **Matrícula:** 202302612 (final 2 → teste adversarial: *mudanças rápidas, ruído ou acionamentos repetidos*) **Grupo/cenário:** o mesmo das Atividades 01 e 02 — balanceamento de fluxo de fila em entrada única de eventos, com validação de ingresso por câmera de celular/tablet e recomendação de fila exibida em painel de LED. **Link do Wokwi:**

<https://wokwi.com/projects/474831849754123265>

1. Recorte escolhido e relação com o projeto do grupo

O grupo modelou, na Atividade 02, um sistema com dois níveis: **dispositivo** (celular/tablet no ponto de validação, que faz a leitura de QR/facial e publica eventos) e **névoa** (servidor local que agrega vazão entre filas e decide a recomendação). Este protótipo individual representa o **lado do dispositivo voltado à comunicação com a névoa**: como o ponto de validação produz e envia eventos, como ele se comporta quando o enlace cai, e como evita duplicar ou perder dados quando volta a se conectar.

Não é implementada a leitura de QR/reconhecimento facial em si (fora de escopo do Wokwi — ver seção 9), nem o serviço agregador da névoa. O botão de validação representa o resultado binário que, na Atividade 02, sai do processamento local do celular; a chave de enlace representa a conectividade MQTT dispositivo↔névoa descrita no item 12 da Atividade 02 (broker inalcançável).

2. Responsabilidade individual assumida: Comunicação e resiliência

Conforme a negociação do grupo, assumi o perfil **Comunicação e resiliência** (contrato de evento, sequência, heartbeat, duplicação, perda, silêncio, desconexão e recuperação). Cada um desses tópicos aparece de forma concreta e verificável no código ([sketch.ino](#)):

Tópico exigido	Onde aparece no protótipo
Contrato de evento	<code>emitEvento()</code> — todo evento sai como JSON com <code>eventType/deviceId/entityId/eventTimeMs/sequence/value/unit/state</code> (seção 3)
Sequência	<code>seqValidacao</code> , <code>hbSeq</code> , <code>seqControle</code> — contadores monotônicos independentes por tipo de evento
Heartbeat	<code>tarefaHeartbeat()</code> a cada 4 s, com <code>bateria_pct</code> , <code>fila_atual_estimada</code> , <code>missedHeartbeats</code>
Duplicação	<code>seqVista()</code> / <code>recordSeq()</code> (anel de 16 sequências) + pressão longa no botão (<code>reinjetarUltimo()</code>) para forçar uma reentrega e observar o descarte
Perda	<code>outboxEnfileira()</code> descarta o item mais antigo quando a fila local enche, mas emite log (<code>perdasOutbox</code>) — nunca descarte silencioso
Silêncio	<code>missedHeartbeats</code> conta ciclos de heartbeat não enviados; após ~3 ciclos (<code>DEGRADO_APOS_MS</code>) a fila é marcada <code>fila.degradada</code>

Tópico exigido	Onde aparece no protótipo
Desconexão	<code>lerEnlace()</code> — máquina de estados <code>LINK_ONLINE</code> / <code>LINK_OFFLINE</code> / <code>LINK_DEGRADADO</code> com debounce de 700 ms
Recuperação	<code>reconectarEFlush()</code> — esvazia a fila local em ordem, aplica dedup evento a evento e emite <code>enlace.recuperado</code> com o resumo

Essa é a diferença verificável em relação a colegas que assumiram outros perfis (ex.: quem ficou com "Decisão e atuação" foca a histerese da regra de negócio; aqui a histerese/debounce está no **enlace**, e o núcleo do código é a fila local + deduplicação, que os outros perfis não precisam implementar).

3. Circuito e componentes

Papel	Componente Wokwi	Pino ESP32	Observação
Validação de ingresso (evento)	wokwi-pushbutton	D4 (INPUT_PULLUP)	Substitui a câmera/QR/facial do celular. Toque curto = uma validação; segurar ~1 s reinjeta manualmente o último evento (simula reentrega de rede)
Enlace dispositivo↔névoa	wokwi-slide-switch	D18 (INPUT)	Substitui o link MQTT dispositivo↔broker. Para 3V3 = ONLINE; para GND = OFFLINE
Bateria do dispositivo	wokwi-potentiometer	D34 (ADC)	Grandeza fictícia (ver seção 9); usada só para popular o campo <code>bateria_pct</code> do heartbeat
Painel de LED — recomendação fresca	LED verde + resistor 220 Ω	D25	
Painel de LED — segurando última recomendação	LED amarelo + resistor 220 Ω	D26	
Painel de LED — fila sem dado confiável	LED vermelho + resistor 220 Ω	D27	
Indicador de fila local pendente	LED azul + resistor 220 Ω	D14	Aceso enquanto há eventos na outbox aguardando reenvio
Alarme de mudança de estado do painel	wokwi-buzzer	D13	Só soa quando o estado do painel muda (evita ruído sonoro contínuo)

Arquivo do circuito: [diagram.json](#). Nenhum componente aqui aciona diretamente a rede real — a simulação de MQTT/broker é feita em software, pela chave de enlace (ver limitações, seção 9).

4. Contrato de evento

Contrato mínimo pedido pela atividade, respeitado em todos os tipos de evento emitidos:

```
{ "eventType": "tipo.do.evento", "deviceId": "identificador-do-dispositivo",
  "entityId": "entidade-observada", "eventTimeMs": 18420, "sequence": 17, "value": 28.7,
  "unit": "C", "state": "ATENCAO" }
```

Tipos de evento produzidos pelo protótipo (todos com `deviceId="esp32-portao2-catraca3"`, `entityId="portao-2-catraca-3"`):

validacao.realizada — cada toque aceito no botão.

```
{"eventType":"validacao.realizada","deviceId":"esp32-portao2-catraca3",
"entityId":"portao-2-catraca-3","eventTimeMs":18420,"sequence":17,"value":1,
"unit":"bool","state":"ENVIADO","resultado":"validado","modo":"qr",
"linkState":"ONLINE","outboxDepth":0,"seqLocal":17}
```

`state` varia entre `ENVIADO` (enlace ok), `ENFILEIRADO` (enlace caído), `REENVIADO` (saiu da fila local após reconexão) e `DUPLICATA_DESCARTADA` (sequência já confirmada).

dispositivo.heartbeat — a cada 4 s, só quando o enlace está online.

```
{"eventType":"dispositivo.heartbeat","deviceId":"esp32-portao2-catraca3",
"entityId":"portao-2-catraca-3","eventTimeMs":20004,"sequence":5,"value":74,
"unit":"%", "state":"OK","bateria_pct":74,"fila_atual_estimada":3,
"missedHeartbeats":0,"bateria_instavel":false,"outboxDepth":0}
```

enlace.perdido, **fila.degradada**, **fila.confivel**, **enlace.recuperado**, **enlace.reinjecao**, **dispositivo.online** — eventos de plano de controle da comunicação (não fazem parte do contrato de negócio, mas seguem o mesmo formato mínimo). Exemplo de recuperação:

```
{"eventType":"enlace.recuperado","deviceId":"esp32-portao2-catraca3",
"entityId":"portao-2-catraca-3","eventTimeMs":45210,"sequence":3,"value":4,
"unit":"evt","state":"RECUPERADO","reenviados":4,"duplicatas_descartadas":1,
"perdas_outbox":0,"offline_ms":26010,"ruído_rejeitado":2,"heartbeats_perdidos":6}
```

`eventTimeMs` é o tempo desde o boot do ESP32 (`millis()`), não um timestamp absoluto — ver limitações (seção 9).

5. Estado e regra (por que não é uma comparação instantânea)

A atuação do painel depende de **três mecanismos de estado/tempo combinados**, nunca de um único valor > limite:

1. **Debounce + histerese temporal no enlace** (`lerEnlace()`): a chave só muda o estado lógico depois de ficar estável 700 ms (`LINK_DEBOUNCE_MS`), e só volta a ser considerada "confiável" depois de 2 s online contínuos (`LINK_ESTAVEL_MS`). Isso evita que uma oscilação rápida da chave produza uma sequência de eventos de conexão/desconexão.
2. **Persistência/expiração da última recomendação** (`atualizarPainel()`): o painel só sai do estado "fresco" quando a última recomendação recebida da névoa fica mais velha que `REC_STALE_MS` (9 s) — uma validade/expiração de dado, não uma leitura instantânea.
3. **Escalonamento por contagem de ciclos** (`lerEnlace()`): a fila só é marcada como "sem dado confiável" depois de ficar offline por mais de `DEGRADO_APÓS_MS` (12 s $\approx$ 3 ciclos de heartbeat).

perdidos) — uma condição de persistência, não um evento único.

Além disso, o próprio evento de validação passa por **debounce mecânico (40 ms) + intervalo mínimo entre eventos aceitos (300 ms)**, e a deduplicação usa um **anel das últimas 16 sequências confirmadas**, ambos mecanismos de estado (não comparações pontuais).

6. Atuação

Estado do painel	LED aceso	Significa
PANEL_FRESCO	Verde (D25)	Enlace online e recomendação recebida há menos de 9 s
PANEL_SEGURANDO	Amarelo (D26)	Enlace caiu ou a recomendação está velha — painel mantém a última recomendação exibida , não apaga nem trava (mesmo comportamento descrito no item 12 da Atividade 02)
PANEL_SEM_CONFIANCA	Vermelho (D27)	Offline há mais de 3 ciclos de heartbeat — a fila seria excluída do cálculo de recomendação da névoa

Regras de segurança/anti-oscilação da atuação:

- LEDs e buzzer só são reescritos **quando o estado muda** (`if ( p != panelAnt )`), nunca a cada iteração do `loop()` — impede comandos repetidos.
- O LED azul (outbox) segue a mesma regra de log/atuação só-em-mudança.
- O buzzer tem um padrão de bipes diferente para cada transição (1 bipe grave = degradando, 2 bipes graves = sem confiança, 2 bipes agudos = recuperado), o que torna a decisão auditável mesmo sem olhar o monitor serial.
- Comportamento explícito com entrada insuficiente: enquanto não há nenhuma recomendação recente e o enlace nunca ficou online (`tRecRecebida` não avançou), o painel permanece coerente em `PANEL_SEGURANDO` / `PANEL_SEM_CONFIANCA` , nunca em um estado indefinido.

7. Teste adversarial (final de matrícula = 2 → "mudanças rápidas, ruído ou acionamentos repetidos")

1) Por que uma implementação ingênua erraria: um código que lê o botão a cada iteração do `loop()` e gera um evento a cada borda de descida contaria múltiplas validações por causa do *bounce* mecânico do botão (típico em simulações de contato); e um código que reage a cada leitura da chave de enlace trocava o painel a cada oscilação, produzindo uma sequência de `enlace.perdido` / `enlace.recuperado` e piscando o painel para o público a cada instabilidade momentânea de rede — exatamente o efeito que a Atividade 02 (item 12) queria evitar.

2) Mecanismo implementado:

- Debounce de 40 ms (`BTN_DEBOUNCE_MS`) no botão + intervalo mínimo de 300 ms entre validações aceitas (`MIN_INTER_MS`); toques mais rápidos que isso são contados em `ruídoRejeitado` e **não** viram evento `validacao.realizada` .
- Debounce de 700 ms na chave de enlace (`LINK_DEBOUNCE_MS`) — só muda de estado depois de ficar parada nesse tempo.

- Histerese de 2 s antes de marcar a fila como "confiável" de novo (`LINK_ESTAVEL_MS`), para não reagir a uma reconexão passageira.
- Deduplicação por sequência (`seqVista / recordSeq`) — mesmo que um evento seja reenviado (reinjeção manual, segurando o botão ~1 s), ele não é contado duas vezes.
- Atuação só muda em transição de estado (seção 6), nunca a cada leitura.

3) Saída observada (execução real no Wokwi):

Botão martelado sem pausa por ~1-2 s (`evidencias/teste-adversarial-botao.png`): entre as validações de `sequence=19` e `sequence=21`, cada uma ficou espaçada 308 ms e depois 329 ms da anterior — sempre acima dos 300 ms mínimos — enquanto o contador de ruído subia continuamente (`total ruido=26` até 29 só nesse trecho de log). Ou seja, da rajada de toques aplicada, a esmagadora maioria virou `RUIDO` rejeitado, sem gerar evento nem incrementar `sequence`:

```
-- [t=41042ms] RUIDO rejeitado: toque a 172ms da ultima validacao (< 300ms) -> NAO vira evento (total ruido=26)
-- [t=41218ms] validacao ACEITA: seq=19 modo=qr (19 na janela de 120s)
-- [t=41365ms] RUIDO rejeitado: toque a 147ms da ultima validacao (< 300ms) -> NAO vira evento (total ruido=27)
-- [t=41526ms] validacao ACEITA: seq=20 modo=facial (20 na janela de 120s)
-- [t=41683ms] RUIDO rejeitado: toque a 157ms da ultima validacao (< 300ms) -> NAO vira evento (total ruido=28)
-- [t=41855ms] validacao ACEITA: seq=21 modo=qr (21 na janela de 120s)
-- [t=41997ms] RUIDO rejeitado: toque a 142ms da ultima validacao (< 300ms) -> NAO vira evento (total ruido=29)
```

Chave de enlace mexida repetidamente por ~1,3 s (`evidencias/teste-adversarial-chave.png`): apareceram 13 linhas `enlace: oscilacao detectada na chave` (foi p/ `OFFLINE`) entre `t=120798ms` e `t=122071ms` (66 a 180 ms uma da outra), e **nenhuma** virou `enlace.perdido` nem mudou o painel. Só quando a chave ficou parada por 700 ms seguidos — a última oscilação foi em `t=122071ms`, e a confirmação veio em `t=122772ms`, exatamente 701 ms depois — o enlace foi considerado `OFFLINE` de verdade, com o `enlace.perdido` correspondente. A recomendação da névoa (seguir fila #1) continuou chegando normalmente no meio da instabilidade (`t=121834ms`), sem ser afetada pela indecisão da chave.

8. Testes obrigatórios

8.1 Estado normal

- **Estado inicial:** enlace `ONLINE` (chave para 3V3), nenhum toque no botão.
- **Sequência de entradas:** nenhuma (apenas aguardar).
- **Valores/tempos relevantes:** heartbeats a cada 4 s; recomendação a cada 5 s.
- **Resultado esperado:** LED amarelo por até ~5 s (nenhuma recomendação real foi recebida ainda), depois LED verde assim que a primeira recomendação chega; sem beeps na transição inicial (`prev == PANEL_BOOT`); `dispositivo.heartbeat` com `state:"OK"` e `missedHeartbeats:0` a cada ciclo; nenhum evento de validação.
- **Resultado observado** (`evidencias/teste-normal.png`): boot com o enlace já `ONLINE`; painel nasceu amarelo (`SEGURANDO`, log "ultima recomendacao ha 9075ms"), sem beep; em `t=2804ms`, `fila.confiable` (2 s de histerese cumpridos); em `t=4807ms` o primeiro heartbeat, com

"state":"OK" e "bateria_pct":97 ; em t=5809ms chegou a primeira recomendação real e o painel virou FRESCO (verde). Em nenhum momento apareceu `validacao.realizada` . Bateu exatamente com o esperado.

- **Explicação:** a regra não atua porque não há condição de negócio satisfeita (nenhuma validação) nem degradação de enlace — comportamento correto de repouso. O LED nasce amarelo (não verde) porque `tRecRecebida` é inicializado já "velho" no boot — corrigido depois que um teste real revelou que a versão anterior mostrava verde por 9 s mesmo com o enlace offline no boot, como se uma recomendação tivesse acabado de chegar sem nunca ter chegado de fato (ver seção 10).

8.2 Decisão e atuação

- **Estado inicial:** enlace ONLINE.
- **Sequência de entradas:** 3 toques espaçados (> 300 ms) no botão; depois colocar a chave em OFFLINE e aguardar; depois religar.
- **Valores/tempos relevantes:** t=0s toque 1 (seq=1, ENVIADO); t=2s toque 2 (seq=2); t=4s toque 3 (seq=3); t=6s chave OFFLINE; t=9s (chave ainda off, rec com >9s) → painel amarelo; t=18s (>12s offline) → painel vermelho + `fila.degradada` ; t=25s chave ONLINE → flush da outbox com os toques dados durante o offline, `enlace.recuperado` ; t=27s (2s estável) → `fila.confivel` ; nova recomendação em até 5s → painel volta a verde.
- **Resultado esperado:** progressão verde → amarelo → vermelho → (flush) → verde, com um evento de validação ENVIADO por toque online e ENFILEIRADO → REENVIADO para os toques dados offline.
- **Resultado observado** (`evidencias/teste-decisao-atuacao.png`): esta captura foi feita com o enlace já OFFLINE desde o boot (antes da correção do item 2 da seção 10), então mostra a metade de degradação: painel nasceu verde (bug já corrigido — ver seção 10), virou amarelo em t=9806ms ("`ultima recomendacao ha 9002ms`"), e em t=12807ms (offline há 12003ms) escalou para vermelho com o evento `fila.degradada` . A metade de reconexão (religar a chave e observar REENVIADO + `enlace.recuperado` + `fila.confivel` + volta ao verde) ainda não foi capturada com o firmware corrigido — os únicos dados reais que tenho dessa fase vêm do Teste 1 (`fila.confivel` em t=2804ms) e do teste adversarial da chave (`enlace.perdido/reconexão` em outro contexto), não deste roteiro específico.
- **Explicação:** a troca de painel depende de estado acumulado (tempo offline, idade da recomendação), não de uma comparação instantânea; a recuperação reprocessa a fila local sem perder nem duplicar eventos.

8.3 Adversarial individual (final 2 — ver seção 7)

- Já detalhado na seção 7 (estado inicial, entradas, esperado/observado e explicação).

9. Limitações da simulação (o que foi validado aqui vs. o que exigiria hardware/ambiente real)

- **Sem RTC/NTP:** `eventTimeMs` é `millis()` desde o boot do ESP32, não um timestamp de calendário/UTC como na Atividade 02 (`tempo_evento` ISO 8601). Em hardware real seria necessário sincronizar com NTP ou usar um RTC.

- **Enlace/MQTT simulados:** não há broker, WiFi, TLS, QoS real nem retransmissão de rede — a chave `swLink` e a lógica local (`linkState`) apenas *representam* essas condições. Uma integração real usaria `WiFi.h` + `PubSubClient` (ou similar), com reconexão automática e $QoS \geq 1$ do broker; isso fica como evolução futura, conforme o "Princípio de escopo" da atividade.
- **Sem reconhecimento facial/QR:** o botão representa apenas o resultado binário `resultado=validado`; nenhuma imagem ou dado biométrico é adquirido, processado ou descartado — não há aquisição real da grandeza "identidade".
- **bateria_pct é fictício:** vem de um potenciômetro, sem qualquer relação com a tensão real de uma bateria física.
- **fila_atual_estimada é um proxy local:** conta validações do próprio dispositivo na janela de 120 s, e não a ocupação real de uma fila física nem a agregação entre múltiplas filas feita pela névoa (isso está fora do escopo deste recorte individual).
- **Persistência apenas em RAM:** a outbox e os contadores são perdidos se o ESP32 reiniciar; um dispositivo real guardaria a fila pendente em memória não volátil (NVS/flash).
- **Temporizações comprimidas:** os intervalos (heartbeat 4 s, recomendação 5 s, degradação em 12 s) foram encurtados frente à Atividade 02 (janela de 120 s, heartbeat a cada poucos segundos) apenas para que o comportamento seja observável em uma sessão de laboratório no Wokwi; a proporção entre os tempos foi mantida.

10. Uso de inteligência artificial generativa

- **Ferramenta:** Claude Code (modelo Claude Sonnet 5, Anthropic).
- **Partes da atividade em que foi usada:** estruturação do sketch (máquina de estados do enlace, fila local/outbox, deduplicação por sequência), geração do `diagram.json` do circuito Wokwi e redação do rascunho deste relatório.
- **Síntese dos prompts/orientações:** "implementar a atividade 3 com base na atividade 2, responsabilidade de comunicação e resiliência, matrícula terminada em 2 (teste adversarial de ruído/acionamentos repetidos)".
- **Sugestões aceitas:** desenho da outbox em anel com descarte visível do item mais antigo; deduplicação por sequência com anel de 16 posições; debounce/histerese separados para o botão e para a chave de enlace; uso de `tone()` nativo do ESP32 em vez de bibliotecas externas de buzzer.
- **Sugestões alteradas/rejeitadas:** cheguei a pedir uma versão em MicroPython (rodando no mesmo ESP32/Wokwi), mas voltei para C++/Arduino porque o fluxo local de simulação em MicroPython no VS Code exige passos extras (licença, firmware genérico, transferir o script via `mpremote` ou colar no REPL) que se mostraram mais frágeis do que o fluxo Arduino/PlatformIO, que compila o `sketch.ino` de verdade em um `.bin` já pronto para o simulador.
- **Erros/limitações encontrados na resposta da ferramenta (todos só apareceram ao rodar de verdade no Wokwi, não no código "no papel"):**
 1. A primeira ideia de projeto considerava conectar o ESP32 a um broker MQTT real dentro do Wokwi; isso é possível apenas de forma limitada (rede simulada, sem controle fino de latência/queda), então a decisão foi simular o enlace por uma chave física controlável — uma limitação de fidelidade documentada na seção 9.
 2. Ao testar de verdade (chave já em OFFLINE no boot), o monitor serial mostrou o painel nascendo **verde** por 9 s antes de virar amarelo — um bug real: `tRecRecebida` era inicializado como

"agora" no `setup()`, então o código tratava "acabei de ligar" como se uma recomendação tivesse acabado de chegar, mesmo nunca tendo recebido nenhuma. Corrigido inicializando `tRecRecebida` já "velha" (ver seção 8.1).

3. No mesmo teste apareceu a linha `E (...) ledc: ledc_get_duty(...): LEDC is not initialized` no serial. A primeira suspeita (o `tone(pin, freq, duracao)` de 3 argumentos, com timer interno de auto-stop) não era a causa real: troquei por uma função `beep()` manual (`tone(pin, freq) + delay() + noTone(pin)`) e o aviso continuou aparecendo — é um aviso interno do driver do ESP32 disparado pela **primeira** chamada a `tone()` / `noTone()` da sessão, não importa a variante usada. Solução final: um "aquecimento" silencioso (`tone(BUZZER, 1000); delay(1); noTone(BUZZER);`) logo no `setup()`, para esse aviso único acontecer no boot em vez de no meio de uma captura de teste.
4. No teste adversarial da chave de enlace, a primeira tentativa (poucos cliques espaçados por ~700-1000ms) não provava nada — cada clique tinha tempo de "assentar" e virava uma troca real, porque o log só mostrava o resultado final (conectado/desconectado), nunca a tentativa em si. Adicionei uma linha de log (`enlace: oscilacao detectada na chave...`) disparada toda vez que a chave muda de posição mas ainda não passou o debounce, tornando a supressão de oscilação **visível** (várias linhas seguidas sem nenhuma confirmação) em vez de depender só da ausência de evento como prova — foi exatamente essa mudança que permitiu capturar a evidência real usada na seção 7.

Eu (Claude) sou responsável por essas correções terem sido feitas e verificadas por recompilação real (`pio run`) a cada mudança; o aluno é responsável por ter acompanhado cada teste na simulação e por validar que as capturas em `evidencias/` refletem o comportamento visto na tela.

11. Checklist antes do envio

- ☒ Protótipo relacionado ao projeto do grupo (seção 1).
- ☒ Responsabilidade individual declarada (seção 2).
- ☒ Circuito executa no Wokwi (link testado e acessível, seção acima).
- ☒ Pelo menos uma entrada (botão) e uma saída observável (LEDs/buzzer).
- ☒ Monitor serial mostra evento JSON com `sequence` (seção 4, com exemplos reais).
- ☒ Regra usa estado/tempo/qualidade, não comparação instantânea (seção 5).
- ☐ Três testes executados e documentados (seção 8) — **estado normal e adversarial** completos com evidência real; **decisão e atuação** só tem a metade de degradação capturada, falta religar a chave e registrar a reconexão (`REENVIADO` / `enlace.recuperado` / `fila.confivel`) com o firmware já corrigido.
- ☒ Teste adversarial corresponde ao final da matrícula (2 → seção 7, com evidência real).
- ☒ Limitações explícitas (seção 9).
- ☒ Uso de IA declarado, incluindo erros encontrados (seção 10).
- ☒ Link do Wokwi preenchido no topo do relatório.
- ☐ ZIP final: `atividade-03-joao-pedro-de-brito-tome-202302612.zip` contendo `sketch.ino`, `diagram.json`, `relatorio.pdf` (exportar este `.md`) e a pasta `evidencias/` com as capturas.